

FPGA Station

1 0 1 1 0 1 0 1 1 1 0 0 1 1 0 1 0 1 0 1 0 1
0 1 1 0 1 1 0 0 0 1 1 1 0 1 0 1 0 0 1 1 0 1 1 1 0
1 1 1 0 0 0 1 1 0 1 0

Hardware Accelerators and much more

COASLESCED TSETLIN MACHINE TRAINING AND INFERENCE

Internship Report
of

Jotshna Tulasi
FS2026002

Indian Institute of Technology, Bhubaneswar
School of Electrical and Computer Sciences
Dept. of Electronics & Communication Engineering

Supervisor: Prof. Dr.Ing Srinivas Boppu
Mentor: K Koushik

July 10, 2026

Abstract

The Coalesced Tsetlin Machine (CoTM) is a rule-based machine learning algorithm, which conducts classification based on propositional logic and binary operations. This internship project is devoted to the hardware implementation of the inference phase of the Coalesced Tsetlin Machine in the Verilog HDL. The training algorithm was also studied and partially implemented. The Inference architecture was implemented and verified on two datasets. The development of the design was conducted using Visual Studio Code, while the simulation and synthesis were conducted using Xilinx Vivado. The inference design provided successful classification on all three datasets, indicating the correctness of the hardware implementation. training stage was only partially implemented using the serial architecture; the parallel version is to be developed in the future work. This research proves the possibility of the Coalesced Tsetlin Machine implementation on FPGA hardware and can serve as the basis for further improvements.

This paper was used as the primary reference for this work Mao et al. (2025).

The complete source code and supporting files developed during this internship are available at GitHub Repository.

1 Introduction

Machine learning (ML) is a branch of artificial intelligence that enables systems to learn patterns from data and perform tasks such as classification and prediction. In recent years, Deep neural networks have achieved excellent performance in many applications. However, they require high computational power, consume more energy, and work as black-box models, making their decisions difficult to understand. The Tsetlin Machine (TM) is a rule-based and interpretable machine learning algorithm that learns logical rules using Tsetlin Automata. It operates on binary inputs and logical operations, making it an efficient and interpretable alternative to neural networks.

Several variants of the Tsetlin Machine have been proposed, including the Vanilla, Regression, Convolutional, and Coalesced Tsetlin Machines. This report focuses on the Coalesced Tsetlin Machine (CoTM), which improves memory utilization and computational efficiency while preserving interpretability. Like most machine learning models, the Coalesced Tsetlin Machine operates in two phases: training and inference. The training phase learns the model from data, while the inference phase uses the trained model to make predictions. These phases are discussed in detail in the following sections.

2 The Inference

Inference is the process of using a trained model to make predictions on unseen data. In the Coalesced Tsetlin Machine, inference is performed by evaluating logical clauses and selecting the class with the highest score. write

2.1 Python Training

Step 1: Select the Training Code: The training and inference codes are implemented in Python. The training code is selected to generate the trained model parameters CAIR (2024).

Step 2: Modify the Training Code: The training code is modified to export the learned clauses and clause weights after training.

Step 3: Organize the Project

The project is organized into Dataset, Notebook, Results, and Sources folders (see Fig. 1). The Notebook folder contains the modified `main.py` file, the Dataset folder stores the `xtrain`, `ytrain`, `xtest`, and `ytest` files, the Results folder stores the generated clause and weight files, and the Sources folder contains all the imported source files, including the `config.py` file, which defines all the implementation parameters.

Step 4: Software Verification: The training code is executed to verify the software accuracy.

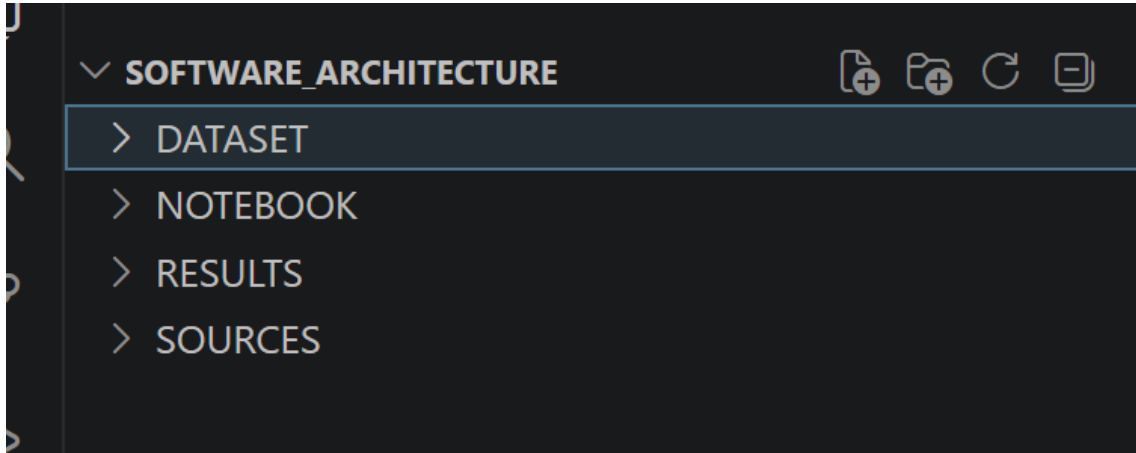


Figure 1: Project Structure

2.2 Inference On Hardware

Step 1: Import Model Parameters The clause and weight memory files generated during Python training are imported into the Verilog implementation.

Step 2: Create Verilog Modules The required Verilog files, including `clauseevaluation.v`, `top.v`, `testbench.v`, `.xdc`, and a parameter file (`.vh`), are created in Vivado.

Step 3: Add Input Files The clause, weight, `xtest`, and `ytest` files are added to the Vivado project.

Step 4: Hardware Implementation Simulation, synthesis, implementation, and bitstream generation are performed, followed by FPGA hardware implementation.

Step 5: Verify Results The hardware accuracy is compared with the software accuracy, and the timing, power, and area reports are analyzed.

2.3 The Noisyxor Dataset

The NoisyXOR dataset is a binary classification dataset consisting of two classes. The Coalesced Tsetlin Machine is configured with 20 clauses, a threshold (T) of 15, specificity (S) of 3.9, 100 Tsetlin Automata states, and is trained for 200 epochs.

Step 1: Generate Model Parameters The Python training code is executed to generate the clause and weight files and obtain the software accuracy (see Figs. 2

Step 2: Perform Verilog Inference The generated clause and weight files are imported into the Vivado project. The hardware performs clause evaluation, class sum computation, ArgMax prediction, and accuracy calculation.

Step 3: Simulation, synthesis, implementation, and bitstream generation are performed in Vivado. The generated bitstream is programmed onto the Basys3 FPGA board to obtain the hardware accuracy.

Step 4: Verify Accuracy Both the software and hardware implementations achieved an accuracy of 87 (see Figs. 3, 4, 5, and 6).

```

main.py  clauses_A.mem  clauses_B.mem
RESULTS > NOISYXOR > clauses_A.mem
1  000000000001000000000000
2  000000100000000000000000
3  000000000100000000000000
4  000000000000000000000000
5  000000001000000000000000
6  001000000000000000000000
7  000000000000100100001110
8  000000001000000000000000
9  001000000000001000000000
10 000010000000100000000000

```

Figure 2: Clauses

```

=====
SIMULATION REPORT
=====
Total Test Vectors Processed : 5000
Correct Class Predictions    : 4376
Final Accuracy               : 87%
Final LED Value              : 0001
=====

```

Figure 3: Hardware accuracy for the NoisyXOR dataset.

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 7.309 ns	Worst Hold Slack (WHS): 0.142 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints:	0
Total Number of Endpoints: 29	Total Number of Endpoints: 29	Total Number of Endpoints:	30
All user specified timing constraints are met.			

Figure 4: Timing Constraint for the NoisyXOR dataset.

2.4 The MNIST Dataset

The MNIST dataset is a multiclass handwritten digit classification dataset with 10 classes and 784 input features. The Coalesced Tsetlin Machine is configured with 200 clauses, 1568 literals, 100 Tsetlin Automata states, and trained for 200 epochs.

The same implementation procedure described for the NoisyXOR dataset is followed for

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	DSPs (90)	Bonded IOB (106)	BUFGCTRL (32)
top module	3126	288	1034	475	1	17	2

Figure 5: Utilisation for the NoisyXOR dataset.

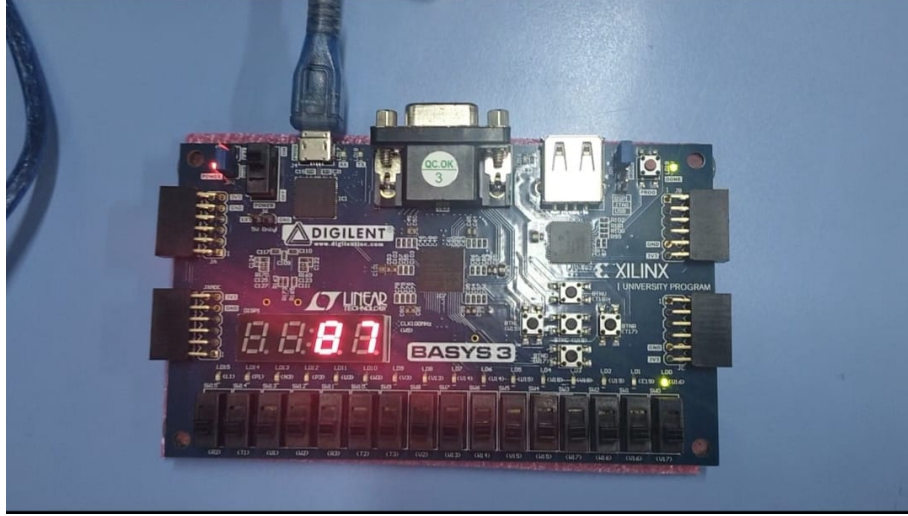


Figure 6: Hardware accuracy for the NoisyXOR dataset on Basys3 Board.

the MNIST dataset. The hardware implementation is performed on the ZCU104 FPGA board, achieving a classification accuracy of 94. (see Figs. 7, 8,9, 10 and 11).

```

=====
      TRAINING SUMMARY
=====
Highest Accuracy : 94.43%
Best Epoch      : Epoch 50
=====

```

Figure 7: Software accuracy for the MNIST dataset.

3 The Training

Training is the process of learning from labeled data by updating the model parameters. In the Coalesced Tsetlin Machine, this is achieved by updating the Tsetlin Automata states and clause weights based on feedback.

The training phase was implemented using a serial architecture. To improve efficiency, a parallel implementation was initiated but remained incomplete during the internship.

```

=====
Correct = 9443
Total   = 10000
Acc      = 94
=====

[TB] =====
[TB] Vectors evaluated : 10000
[TB] Correct           : 9443
[TB] Final Accuracy    : 94 %
[TB] =====

```

Figure 8: Hardware accuracy for the MNIST dataset.

Name	CLB LUTs (230400)	CLB Registers (460800)	CARRY8 (28800)	F7 Muxes (115200)	F8 Muxes (57600)	Block RAM Tile (312)	Bonded IOB (360)	HPIOBDIFFINBUF (192)	BUFGCE (208)
top_module	28310	547	106	10026	839	0.5	8	1	1

Figure 9: Utilization report of MNIST dataset

The complete training process, previously implemented in Python, is implemented in Verilog so that both training and inference can be performed entirely in hardware

3.1 Serial Implementation Results

The serial implementation does not achieve the desired performance but serves as a proof of concept for the hardware implementation of the training process.(see Figs. 12 and 13).

The serial implementation was evaluated using 10 training samples and 50 epochs, achieving an accuracy of 70. Due to the sequential nature of the implementation, scaling to the complete dataset (5000 samples and 200 epochs) was impractical because of the increased hardware resources and execution time. Therefore, a parallel implementation is preferred for efficient training on larger datasets.

4 Conclusion

This internship focused on the hardware implementation of the Coalesced Tsetlin Machine using Verilog HDL. The inference phase was successfully implemented and verified on the NoisyXOR and MNIST datasets. The hardware implementations on the Basys3 and ZCU104 FPGA boards achieved accuracies comparable to the software implementation, validating the correctness of the proposed design.

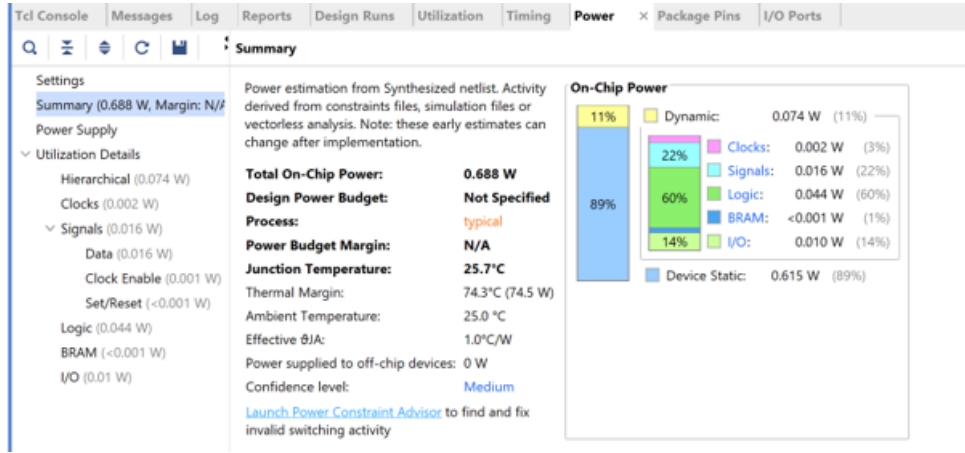


Figure 10: Power report for the MNIST dataset.

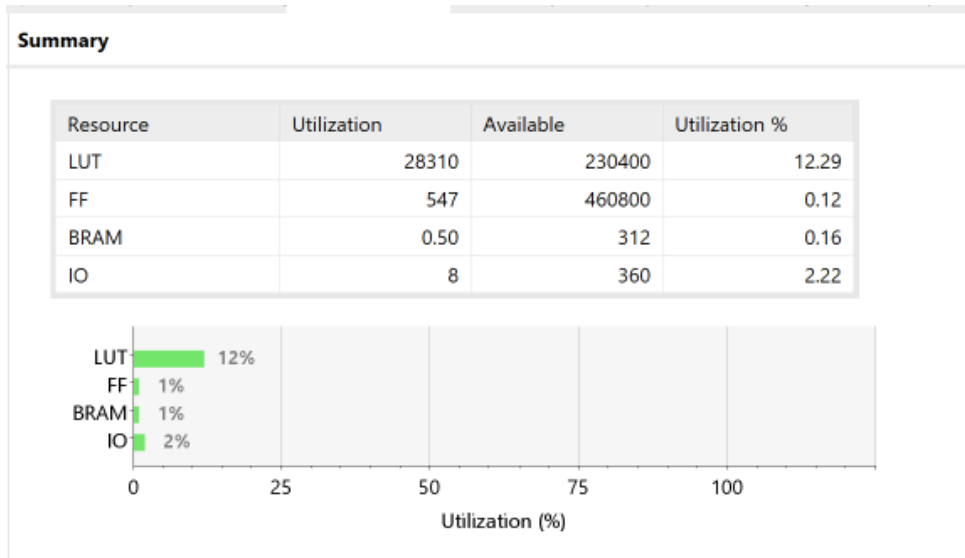


Figure 11: Summary report for the MNIST dataset.

The training phase was also explored through a serial hardware implementation of the Tsetlin Automata state and clause weight update process. While the serial implementation successfully demonstrated the training workflow, it was not suitable for large datasets due to its high resource utilization and execution time. To address this limitation, work was initiated on a parallel hardware architecture. However, its implementation could not be completed within the internship period.

Overall, this internship provided valuable experience in FPGA-based hardware design and the implementation of interpretable machine learning algorithms. The work demonstrates the feasibility of implementing the Coalesced Tsetlin Machine on FPGA platforms and lays the foundation for developing an efficient parallel hardware architecture for both training and inference in future work.

```

=====
LEARNED CLAUSES
=====
Clause 0 : 00000000000000000000000000000000
Clause 1 : 00000010000000000000000000000000
Clause 2 : 00000000000000000000000000000000
Clause 3 : 00000000110100000000000000000000
Clause 4 : 00000100111100000000000000000000
Clause 5 : 00001001000000000000000000000000
Clause 6 : 01000000000000000000000000000000
Clause 7 : 00110000010000000000000000000000
Clause 8 : 00010010000000000000000000000000
Clause 9 : 00000001000000000000000000000000
Clause 10 : 00000000001010000000000000000000
Clause 11 : 01000100001000000000000000000000
Clause 12 : 00010010100000000000000000000000
Clause 13 : 01011011011000000000000000000000
Clause 14 : 00000110000110000000000000000000
Clause 15 : 01001110000000000000000000000000
Clause 16 : 00111001111110000000000000000000
Clause 17 : 00000000010000000000000000000000
Clause 18 : 00001000000000000000000000000000
Clause 19 : 00000000001000000000000000000000

=====
CLAUSE WEIGHTS
=====
Class 0 :  -1  -4  -3  -2  -3  -1  4  0  -4  0  -1  -6  -7  -5  -4  -3  -4  0  1  -1
Class 1 :  10  15  13  15  15  15  10  15  14  15  15  14  15  15  15  15  13  15  13  15

```

Figure 12: Serial training implementation results.

References

- CAIR. (2024). *Tmu: Tsetlin machine library*. <https://github.com/cair/tmu/blob/main/examples/classification/MNISTDemoCoalesced.py>. (GitHub repository)
- Mao, G., Rahman, T., Maheshwari, S., Pattison, B., Shao, Z., Shafik, R., & Yakovlev, A. (2025). Dynamic tsetlin machine accelerators for on-chip training at the edge using fpgas. *arXiv preprint arXiv:2504.19797*. Retrieved from <https://arxiv.org/abs/2504.19797> doi: 10.48550/arXiv.2504.19797

```
=====
FINAL RESULTS
=====
Correct Predictions : 7
Total Predictions   : 10
Accuracy            : 70.00 %

=====
SIMULATION FINISHED
=====
```

Figure 13: Serial training implementation accuracy